

操作系统，2026

第14讲 文件系统(2)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

文件系统的实现

02

共享文件

03

磁盘空间管理

04

文件系统的可靠性

05

文件系统实例

文件系统的实现

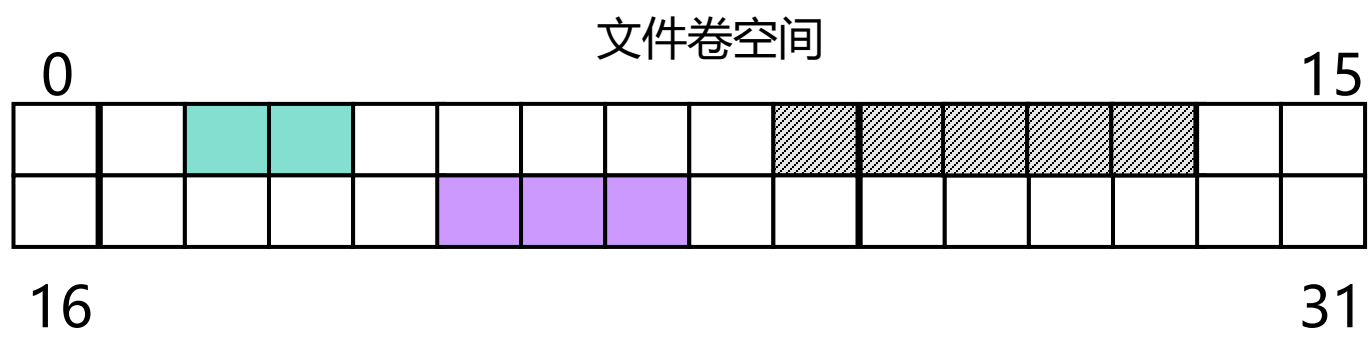
- 文件的实现
- 目录的实现

文件的实现

- 从系统的角度来看文件，从文件在物理介质上的存放方式来研究文件——文件的物理结构
 - 连续分配
 - 链接分配
 - FAT分配
 - 索引分配

连续分配

- 文件存放在若干连续的簇(磁盘块)中
- 目录项中只需存放首簇的地址和簇数



目录		
文件名	起始地址	大小
Hello.c	2	2
z1.cc	9	5
a.out	21	3

连续分配

■ 优点:

- 简单，易实现
- 支持顺序存取和随机存取
- 顺序存取速度快
- 所需的磁盘寻道次数和寻道时间最少

■ 缺点:

- 文件不能动态增长
- 外部碎片问题

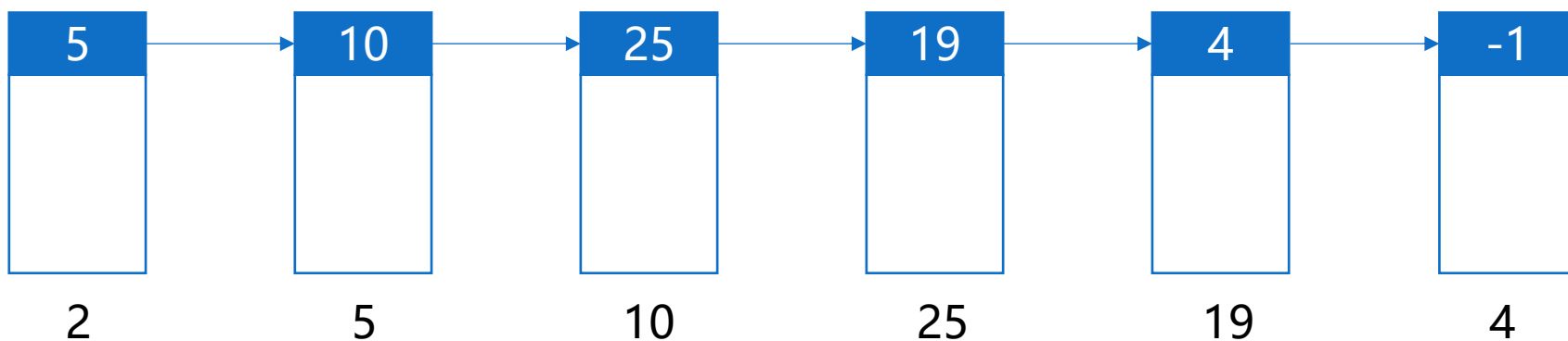
连续分配

- 在CD-ROM文件系统中广泛使用
 - 文件大小事先已知
 - 在使用中文件大小不再改变



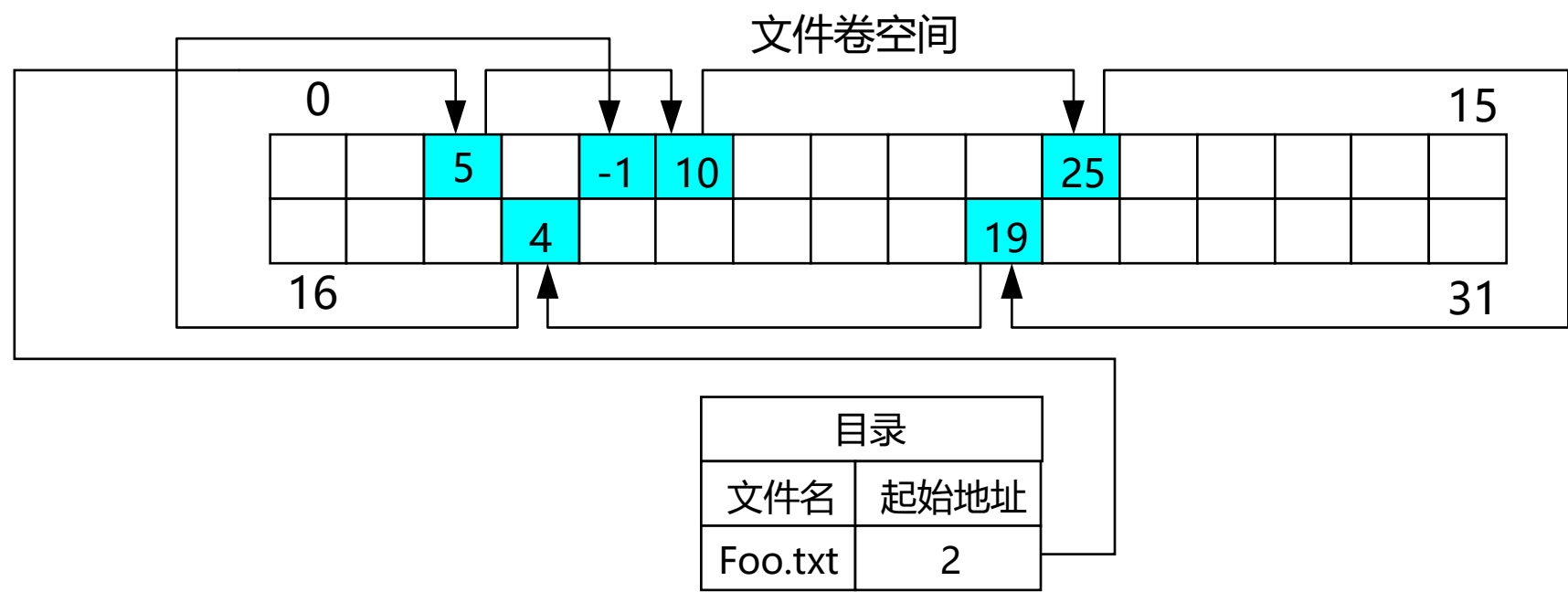
链接分配

- 一个文件的信息存放在若干不连续的簇中，各簇之间通过指针连接，前一个簇指向下一个簇
- 每个簇中包含1个指针字段，其余部分存放数据



链接分配

- 在目录项中，只要存放首簇的地址
- 文件的其他簇可以通过簇中的指针得到



链接分配

■ 优点

- 充分利用磁盘空间，没有外碎片
- 顺序访问速度快

■ 缺点

- 随机访问性能差
- 较多的寻道次数和寻道时间
- 由于指针占去一些字节，数据部分不再是2的整数幂次

FAT分配

- 链接分配的一种实现方式，克服数据部分不是2的整数幂次的缺点
- 取出每个簇中的指针字段，存放在一个单独的表中，称为文件分配表 (File Allocation Table, FAT)
- 在目录中只需记录文件的起始簇号，其余的簇可以从FAT表中获得



FAT分配

■ 优点

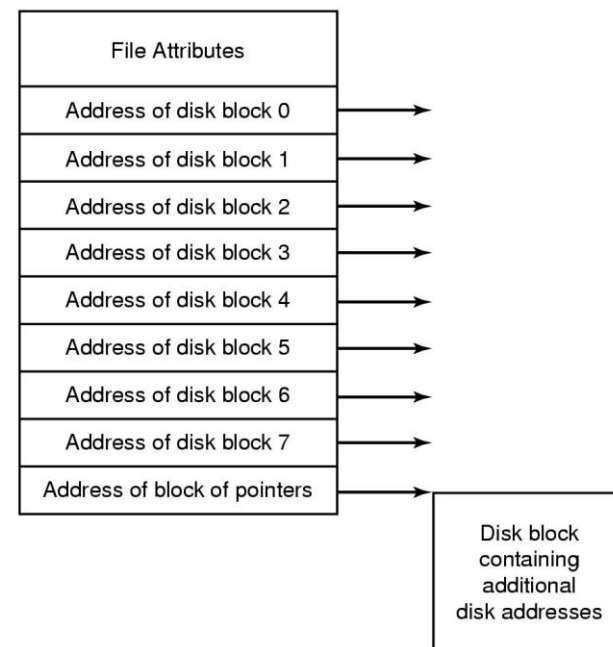
- 充分利用磁盘空间，没有外碎片
- 顺序访问速度快

■ 缺点

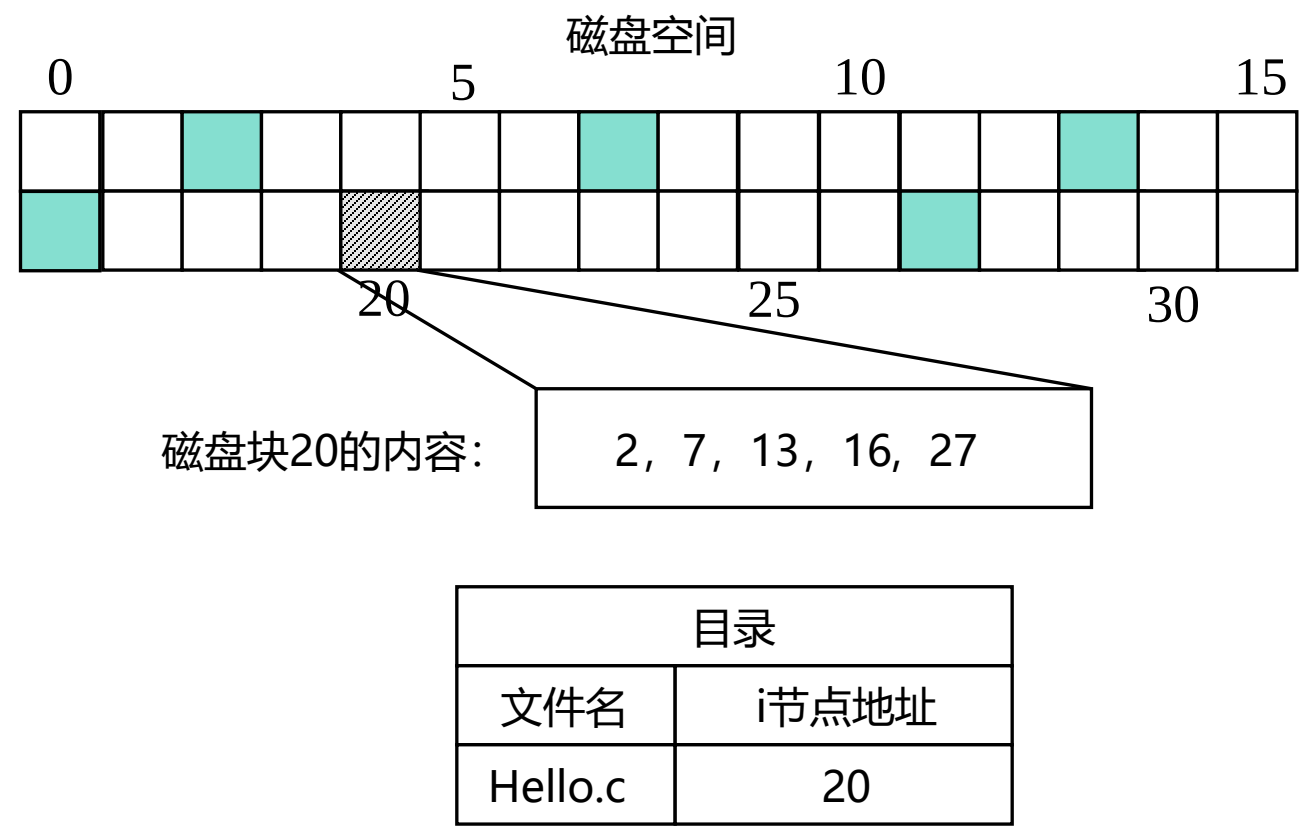
- 随机访问性能差
- 较多的寻道次数和寻道时间
- FAT本身占用一定的磁盘空间

索引分配

- 一个文件的信息存放在若干不连续的磁盘块(簇)中，系统为每个文件建立一个专用数据结构——**i节点(索引节点)**，并将这些块的块号存放在一个i节点中
- 一个i节点就是磁盘块地址数组，其中第n个条目指向文件的第n块
- 当一个文件包含的磁盘块数超出i节点所能容纳的数目时，最后一个项不是指向数据块，而是指向包含附加磁盘地址的磁盘块



索引分配



索引分配

■ 优点

- 即能顺序存取，又能随机存取
- 满足了文件动态增长、插入删除的要求
- 能充分利用外存空间

■ 缺点：

- 较多的寻道次数和寻道时间
- 索引表本身带来了系统开销

目录的实现

- 目录是目录项的有序集合
- 目录项中包含的信息
 - 文件名
 - 查找文件磁盘块所需要的信息
 - 是否包含文件属性则随不同的文件系统而不同

目录的实现

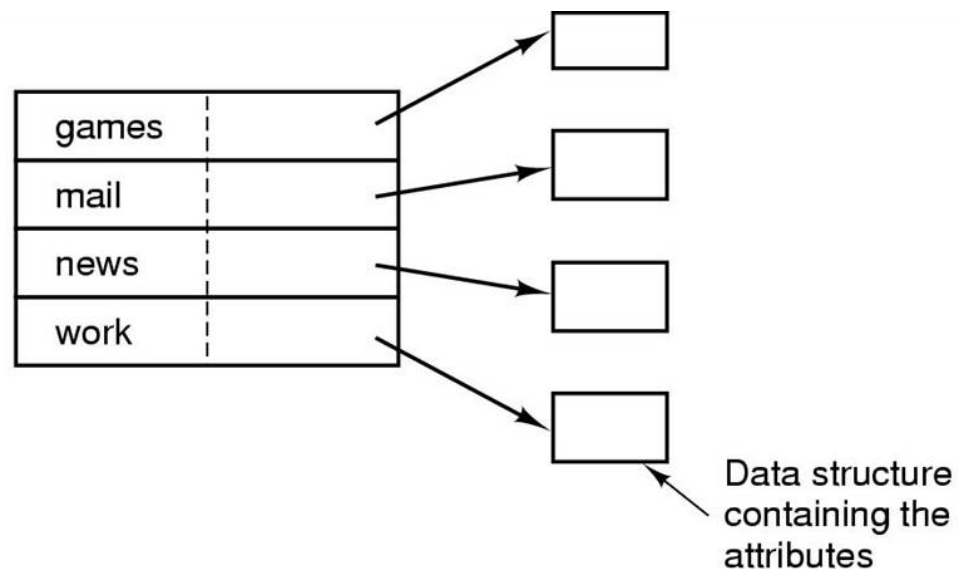
- 简单目录：文件属性直接存放在目录项中

games	attributes
mail	attributes
news	attributes
work	attributes

目录的实现

■ 目录项只引用i节点的目录

- 对于采用i节点的系统，文件属性存放在i节点中而不是目录项中



变长文件名存储

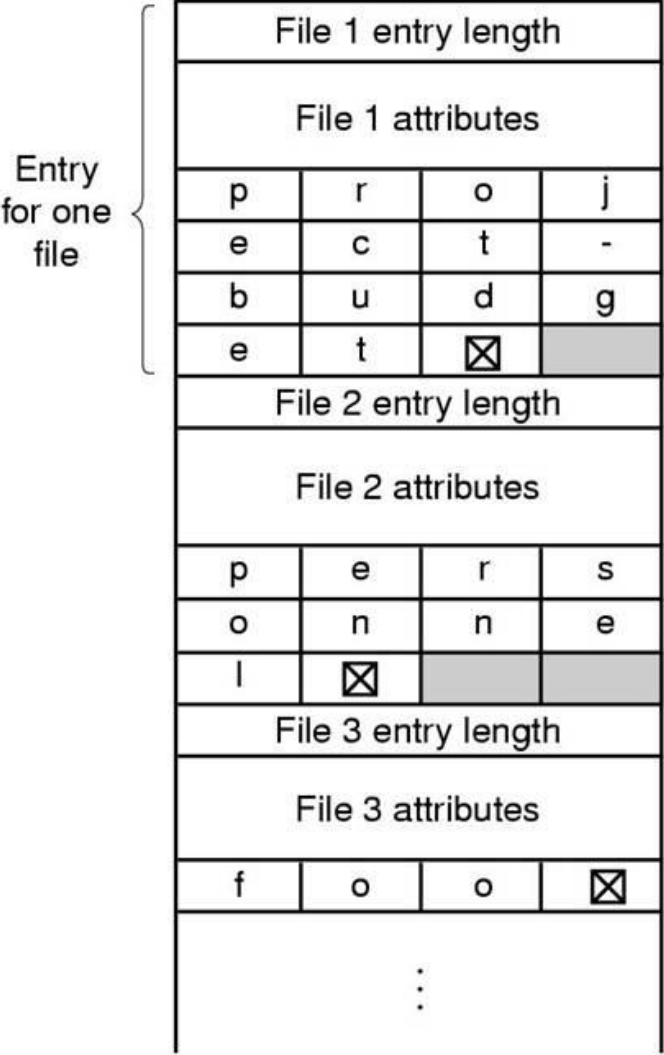
■ 在行中

■ 每个目录项包含固定部分和可变部分

- 固定部分：目录项长度、文件属性
- 可变部分：实际文件名(填充成整数个字)

■ 缺点：

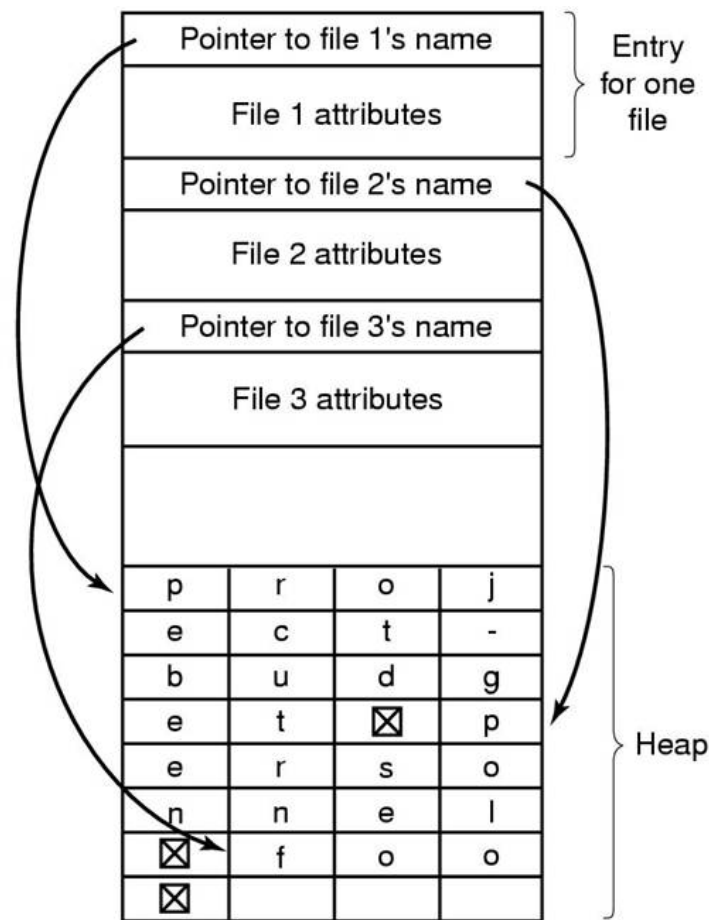
- 删除文件后，留下可变长度的空隙
- 一个目录项可能分布在页面的边界上



变长文件名存储

■在堆中

- 每个目录项都有固定的长度，文件名放置在目录后面的堆中
- 优点：
 - 当一个目录项被删除后，另一个目录项总是可以适合这个空隙
- 缺点：
 - 必须对堆进行管理



下列说法中，_____不属于文件的物理结构的范畴。

- ☐ A 连续文件
- ☒ B 系统文件
- ☐ C 索引文件
- ☐ D 链接文件

提交

CD-ROM文件系统适用于_____。

- ☒ A 连续分配
- ☐ B 链接分配
- ☐ C 索引分配
- ☐ D 以上均不适合

提交

大纲

01

文件系统的实现

02

共享文件

03

磁盘空间管理

04

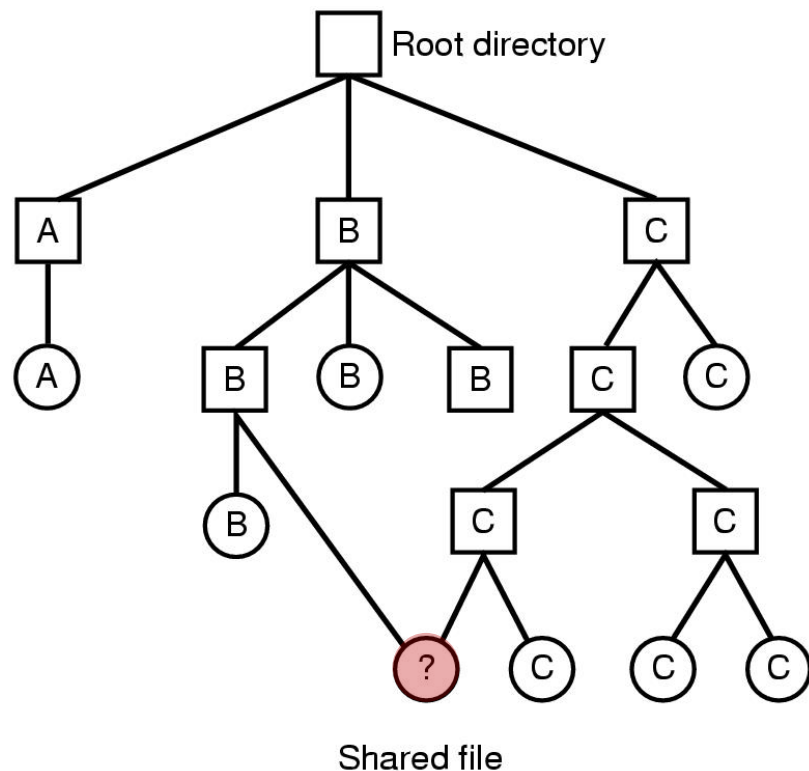
文件系统的可靠性

05

文件系统实例

共享文件

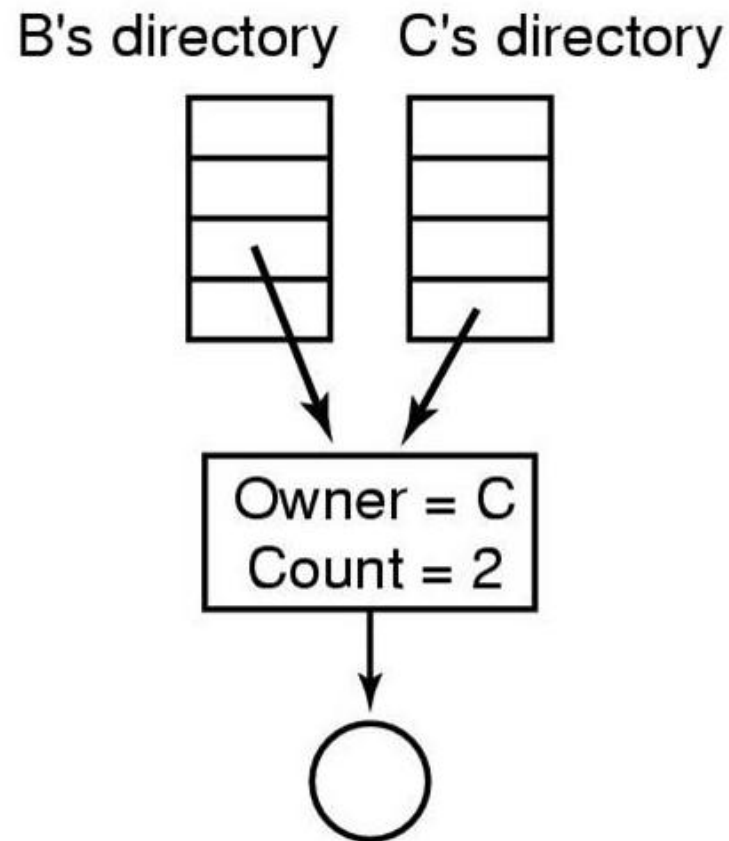
- 多个用户协同工作时，常常需要共享文件
- 共享文件同时出现在属于不同用户的目录下
- 实现共享文件的两种方式：硬链接和符号链接



硬链接

- 目录项指向i节点
- i节点包含磁盘数据块的信息
- 本质是为同一个文件起了一个新别名

ln /源路径/源文件 /目标路径/硬链接名



硬链接

■ 优点

- 不占用额外空间。硬链接只是在目录中增加了一条记录指向现有inode，不分配新的inode，也不占用实际的数据块
- 访问速度快，没有性能损耗。访问硬链接和访问源文件的速度完全相同

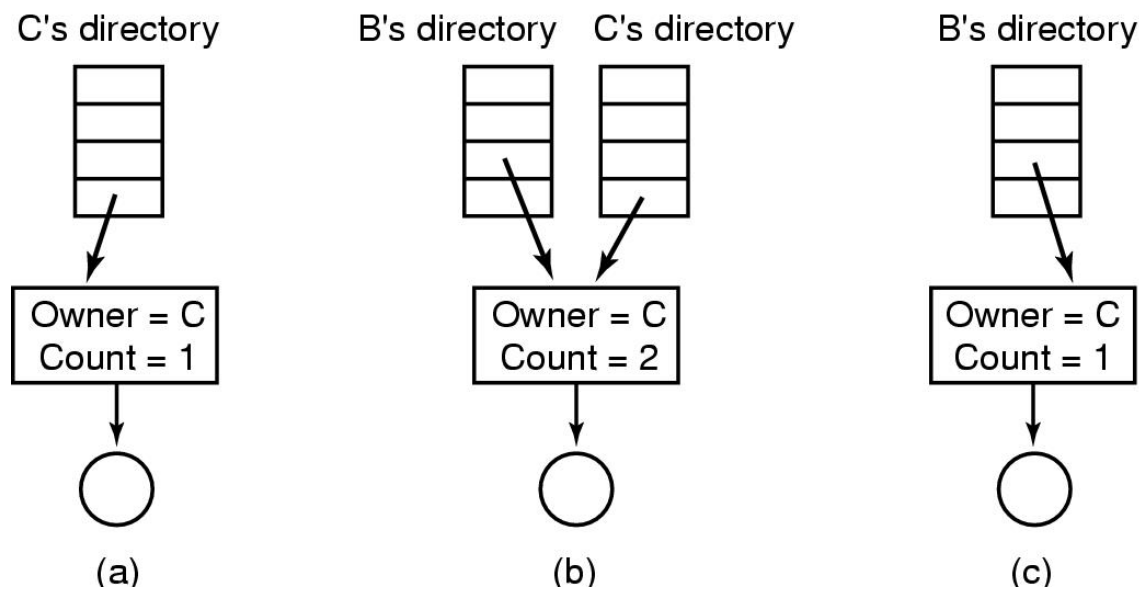
■ 缺点

- 不能跨文件系统，因为inode在不同的文件系统之间是独立的
- 不允许指向目录，否则可能在文件系统中形成死循环
- 硬链接和源文件共享同一个inode，它们的权限、所有者、修改时间等完全相同，无法独立设置

硬链接

■ 多用户系统中硬链接的问题

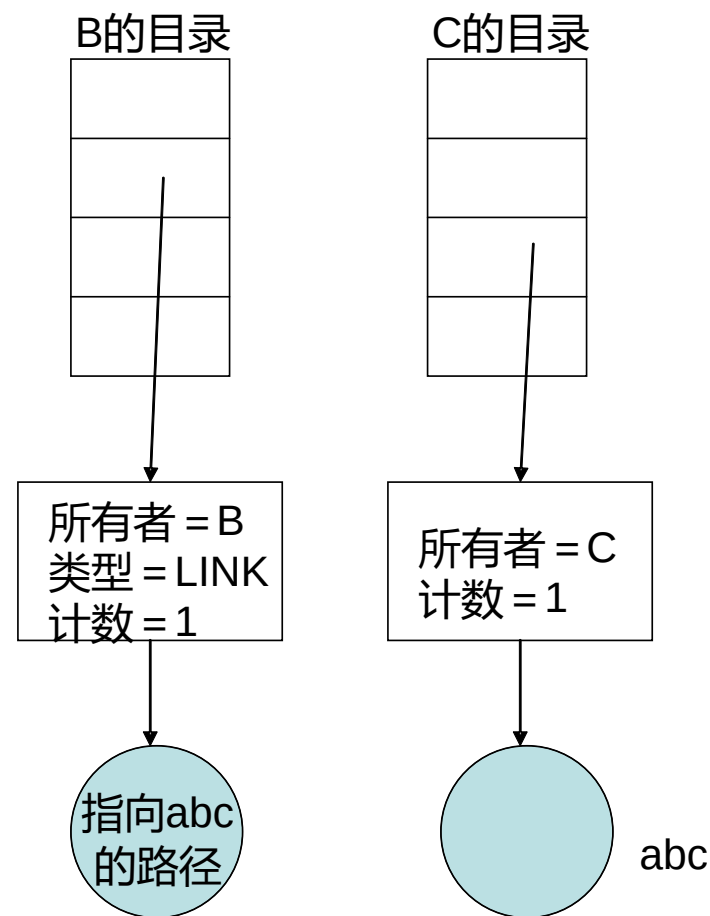
- 假设文件是C创建，B在自己的目录下创建了该文件的硬链接
- C删除文件时，因为计数不为零，所以只删除目录项，不删除i节点和文件本身
- 文件的所有者仍为C，C继续为文件付帐



符号连接

- 类型为LINK的独立文件，该文件只包含它所连接的文件的路径名

`ln -s /源路径/源文件 /目标路径/符号链接名`



符号链接

■ 优点

- 使用灵活，可以跨文件系统、甚至跨网络共享文件
- 拥有独立的inode，可以有独立的权限和属性设置
- 允许指向目录（潜在的死循环可以检测）

■ 缺点

- 容易断链。如果源文件被删除或者重命名，软链接就会失效
- 系统在访问软链接时，需要先读取链接文件获取路径，再根据路径去寻找目标文件，这个二次解析的过程存在轻微的性能损耗
- 因为符号是一个独立的文件，所以会占用一个inode以及存储路径字符串的少量磁盘空间

大纲

01

文件系统的实现

02

共享文件

03

磁盘空间管理

04

文件系统的可靠性

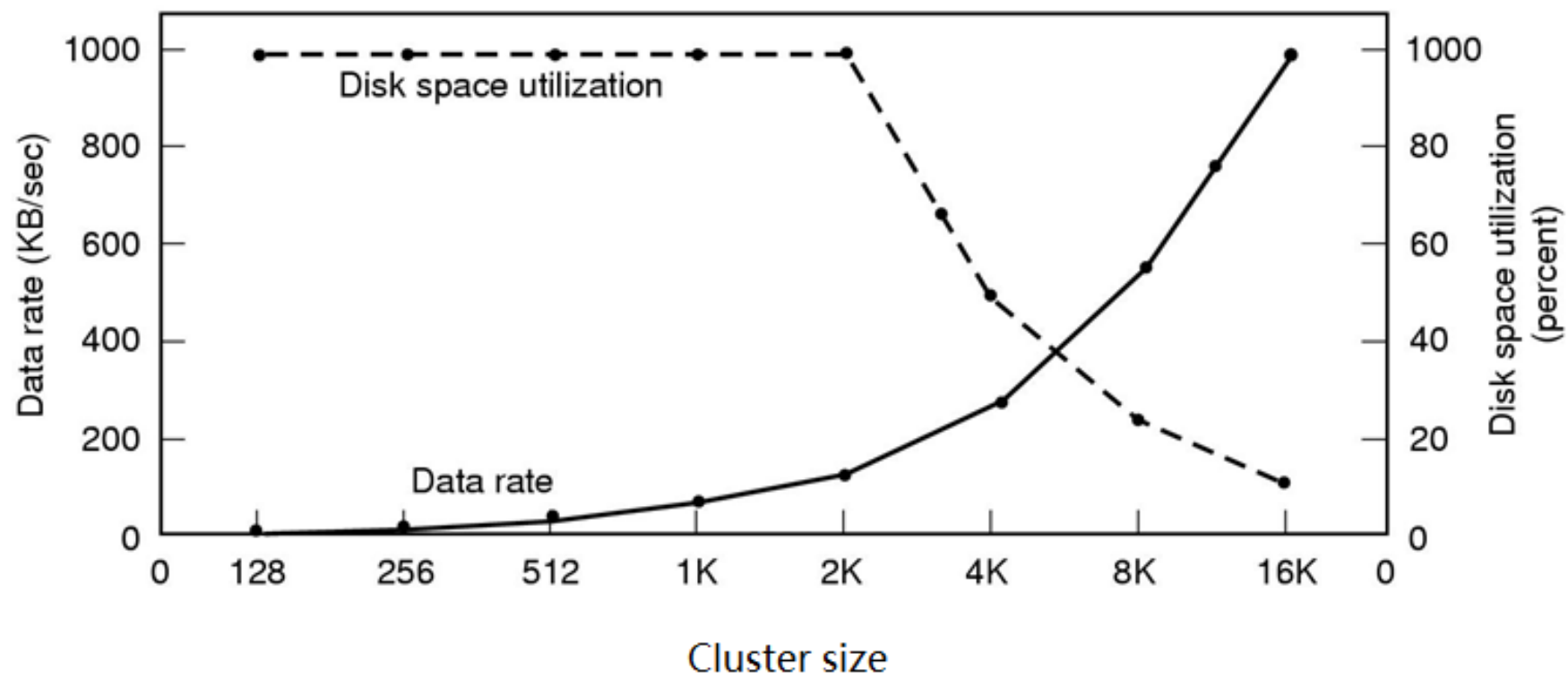
05

文件系统实例

簇的大小

- **两个极端**：大到能容纳整个文件，小到只有一个外存扇区
- **簇较大**：提高I/O访问性能，减小管理开销；但簇内碎片浪费问题较严重
- **簇较小**：簇内的碎片浪费较小，特别是大量小文件时有利；但可能存在簇编号空间不够的问题(如FAT12、16、32)

簇的大小



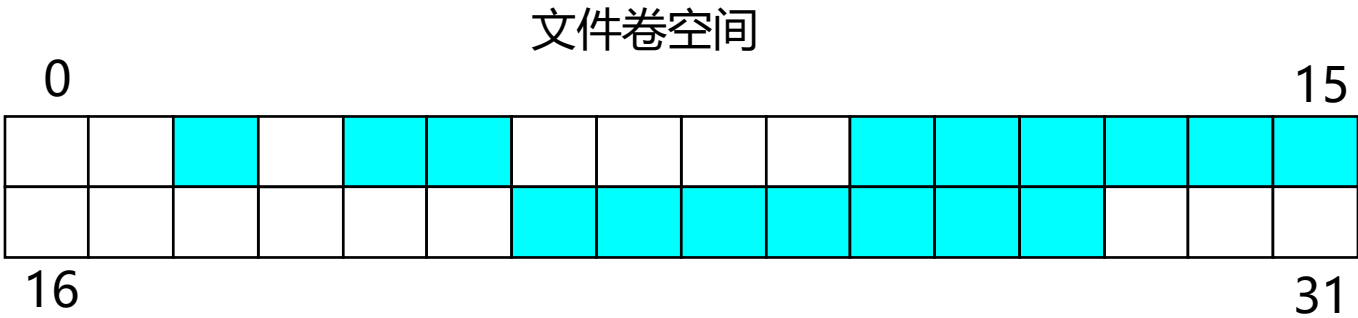
文件卷容量与簇大小的关系

- 若簇的总数保持不变即簇编号所需位数保持不变，文件卷容量越大，则簇越大
 - 缺点：簇内碎片浪费越多
- 若簇大小不变，文件卷容量越大，则簇总数越多，相应簇编号所需位数越多
 - 缺点：可能导致簇编号空间不够

空闲空间管理

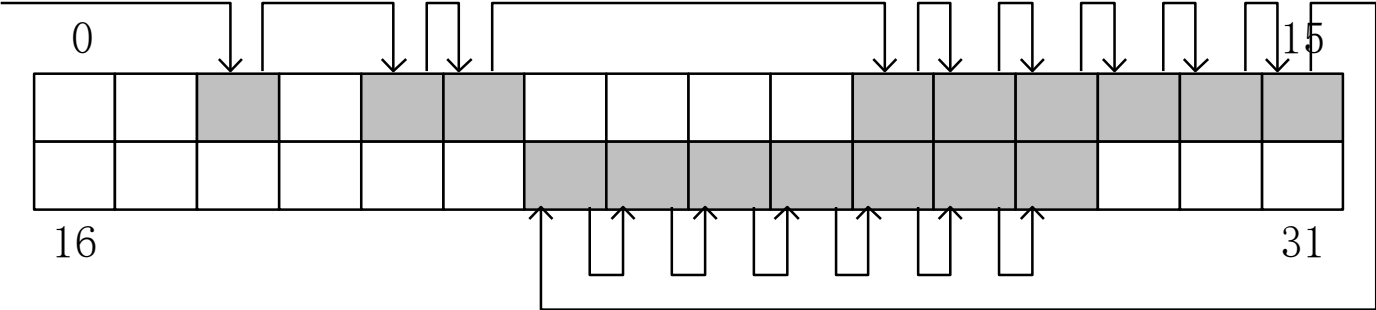
空闲块位图

00101100001111110000001111111000



空闲空间管理

空闲块链表



大纲

01

文件系统的实现

02

共享文件

03

磁盘空间管理

04

文件系统的可靠性

05

文件系统实例

文件系统的可靠性

- **可靠性**——系统抵抗和预防各种物理性破坏和人为性破坏的能力
- 比起计算机的损坏，文件系统破坏的损失要大得多

文件备份

- 文件备份主要是为了解决两个问题：
 - 从意外的灾难中恢复
 - 从错误的操作中恢复
- 文件备份通常是将磁盘文件转储到磁带上保存。转储磁盘到磁带上有两种方案：
 - 物理转储
 - 逻辑转储

物理转储

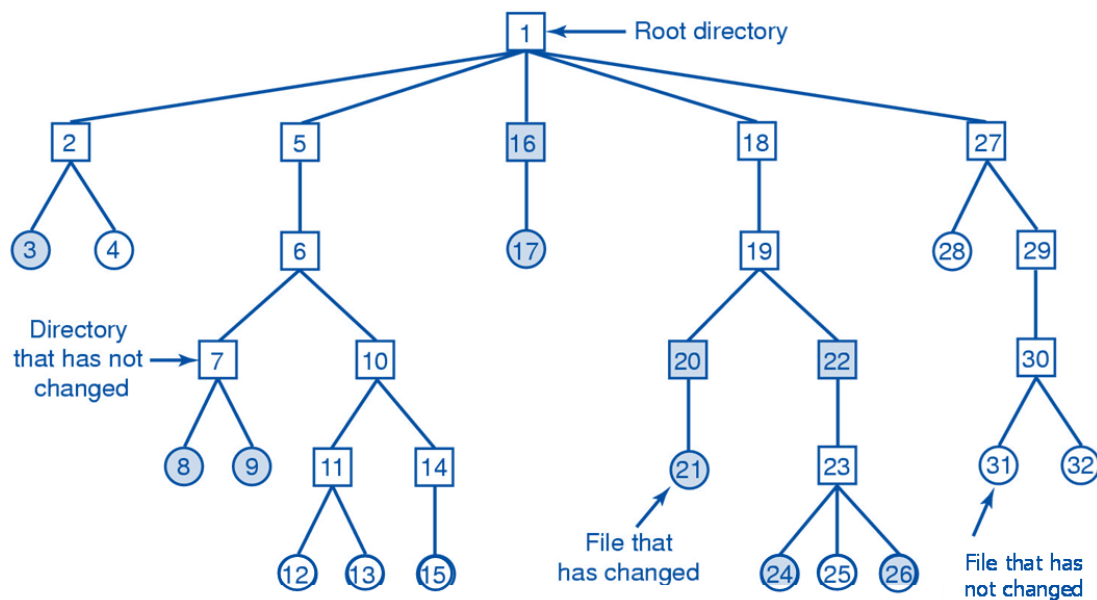
- 从磁盘的第0块开始，将全部的磁盘块按顺序输出到磁带上
- 优点：简单
- 缺点：不能跳过选定的目录、不能实现增量转储、不能恢复单个的文件
- 增量转储：周期性地做全面备份，而每次只对最近更改的数据做备份

逻辑转储

- 从一个或几个指定的目录开始，递归地转储自给定基准日期后有所更改的全部文件和目录
- 优点：很容易满足恢复特定文件或目录的请求
- 缺点：实现复杂

逻辑转储

- 被阴影覆盖的项目代表自基准日期以来修改过，因此需要转储，无阴影的不需要转储

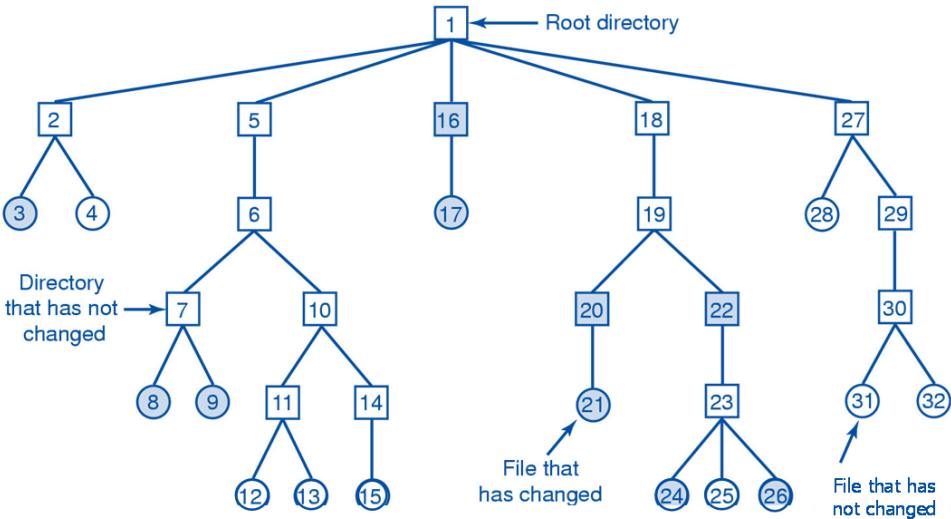


逻辑转储

- 需要转储通向修改过的文件或目录上的所有目录(即使其本身未修改过)
 - 方便将这些转储的文件和目录恢复到另一台计算机上
 - 可以对单个文件进行增量恢复

逻辑转储

■ 算法的执行分四个阶段



(a) 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

(b) 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

(c) 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

(d) 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

文件系统的一致性

- 文件的修改操作：读取磁盘块，修改，写回磁盘
- 如果修改过的磁盘块全部写回之前系统崩溃，则文件系统可能处于不一致状态
- 文件系统一致性检查
 - 块的一致性检查
 - 文件的一致性检查

块的一致性检查

- 构造两张表，每张表中为每个块设立一个计数器。第一个表中的计数器跟踪该块在文件中出现的次数；第二个表中的计数器跟踪该块在空闲表中出现的次数
- 两张表每个块对应的计数初值为0
- 检验程序首先读取全部i节点，得到相应文件使用的块的块号。每当读到一个块时，第一个表中的计数值增1；然后检验程序检查空闲表，找到全部未使用的块，并使第二个表中对应的计数值增1

块的一致性检查

■一致的情况

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	Blocks in use
0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	Free blocks

块的一致性检查

■ 块丢失的情况

- 不会造成大的损害，但是浪费磁盘空间
- 解决方法：加到空闲表中

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	Blocks in use
0	0	0	0	1	0	0	0	0	1	1	0	0	0	1	1	Free blocks

块的一致性检查

- 空闲表中有重复块的情况
 - 解决方法：重建空闲表

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	Blocks in use
0	0	1	0	2	0	0	0	0	1	1	0	0	0	1	1	Free blocks

块的一致性检查

■ 重复数据块的情况

- 解决方法：分配一个空闲块，将块5的内容复制到其中，并将该块插入到一个文件中。这样虽然可以保证文件系统的一致性，但是错误并未消除，因此应该报告这一错误

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	1	0	1	0	2	1	1	1	0	0	1	1	1	0	0	Blocks in use
0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	Free blocks

文件的一致性检查

- 构造一张计数器表，为每个文件设立一个计数器，每个文件对应的计数初值为0
- 检验程序从根目录开始检验，对每个目录中的每个文件，将文件计数器增1
- 检查完成后，得到一张由i节点索引的表，说明每个文件被包含在多少个目录中。然后检验程序将计数值与i节点中的连接数进行比较

文件的一致性检查

- 两个计数值相等 $\Rightarrow$ 文件系统一致
- i 节点的连接数 $>$ 目录项数
 - 可能存在不属于任何目录的文件从而浪费磁盘空间
 - 修改 i 节点中的连接计数
- i 节点的连接数 $<$ 目录项数
 - 存在潜在的灾难
 - 修改 i 节点中的连接计数

关于符号连接，下述叙述**错误**的是_____。

- ☐ A 符号连接是一种特殊的文件类型
- ☒ B 符号连接的i节点与被连接的文件相同
- ☐ C 符号连接包含被连接的文件的路径名
- ☐ D 符号连接存在额外开销

提交

计算机系统崩溃后通常要进行文件系统的一致性检查，文件系统的一致性检查不会进行的工作是_____。

- ☐ A 索引节点重建
- ☐ B 空闲空间重新组织
- ☒ C 文件备份
- ☐ D 错误报告

提交

位图可用于_____。

- ☐ A 文件目录的查找
- ☒ B 文件系统空闲空间管理
- ☐ C 文件的共享
- ☐ D 文件的保护

提交

大纲

01

文件系统的实现

02

共享文件

03

磁盘空间管理

04

文件系统的可靠性

05

文件系统实例

文件系统实例

- FAT
- NTFS——Windows
- EXT——Linux

FAT文件系统

- FAT(File Allocation Table, 文件分配表)文件系统属遗留文件系统。为了向后兼容, 也为了方便用户升级, Windows仍然提供对FAT的支持
- 每一种FAT文件系统都用一个数字来标识磁盘上簇号的位数。例如, FAT12的簇标识为12位(二进制数), 这限制了它的单个分区最多只能存储 $2^{12}(=4096)$ 个簇, 而FAT12在Windows中的簇大小在512B与8KB之间, 这意味着FAT12卷的大小至多只有32M
- 支持多级目录, 无用户访问权限控制

FAT文件系统

■每一种FAT文件系统都用一个数字来标识磁盘上簇号的位数:

■ FAT12/FAT16

■ FAT32

系统	文件分配表中每个簇的字节数	簇界限
FAT12	1.5	小于4087簇
FAT16	2	介于4087和65526簇之间(包括边界)
FAT32	4	介于65526和268,435,456簇之间(包括边界)

FAT文件系统

■卷结构

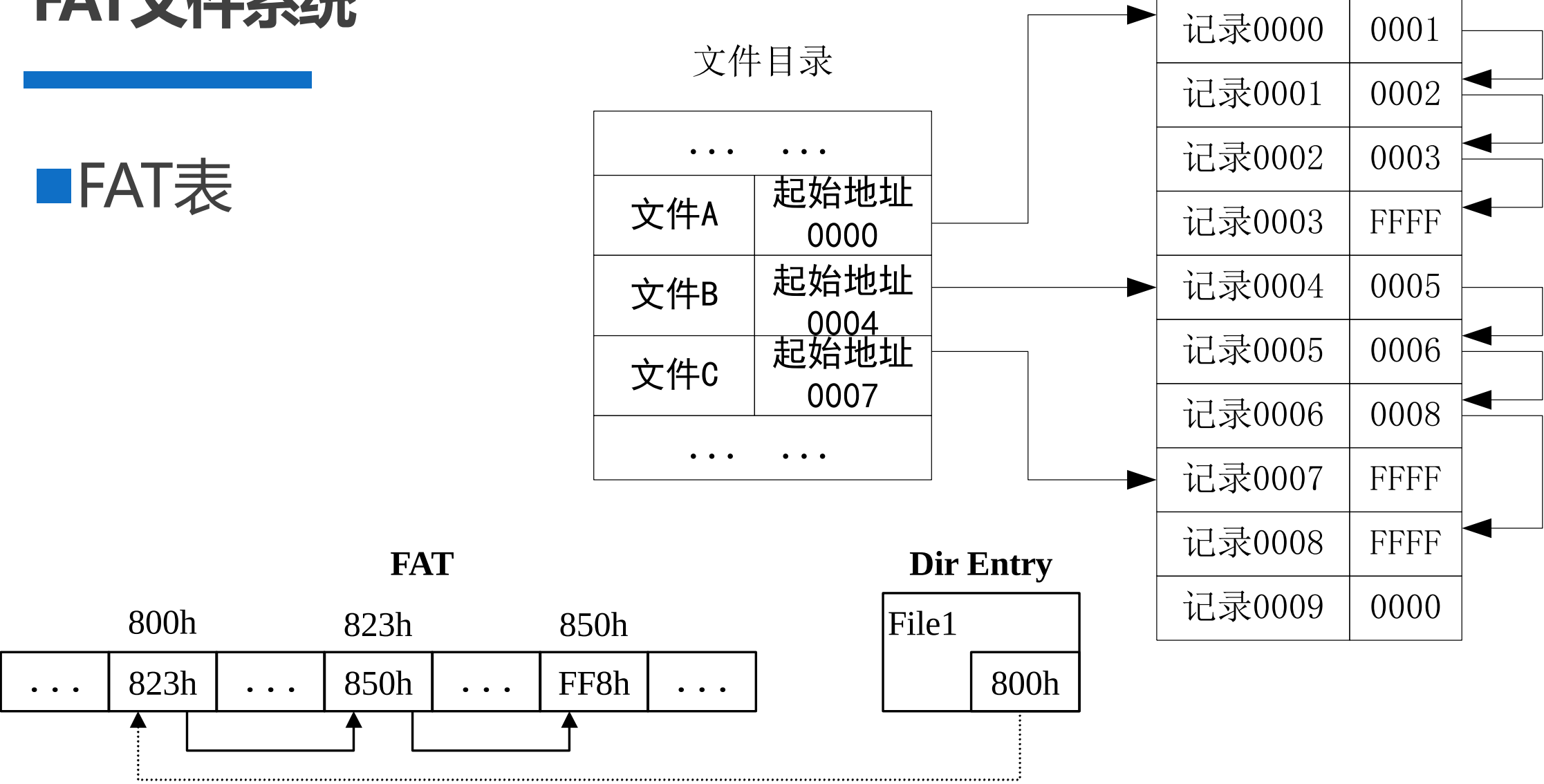
引导区	文件分配表1	文件分配表2	根目录	其他目录和文件
-----	--------	--------	-----	---------

FAT文件系统

- FAT表两个镜像，互为备份。文件卷中的每个簇均对应一个FAT表项，文件分配采用链式分配方法
- 每个FAT表项所占位数是簇编号的位数，其值是(以FAT12为例):
 - 0：表示该簇空闲
 - FF7h：物理坏扇区
 - FF8h~FFFh：表示该簇是文件的最后一个簇
 - 其他值：表示该簇被文件占用，而且表项中的值是文件下一个簇的编号

FAT文件系统

FAT表



FAT文件系统

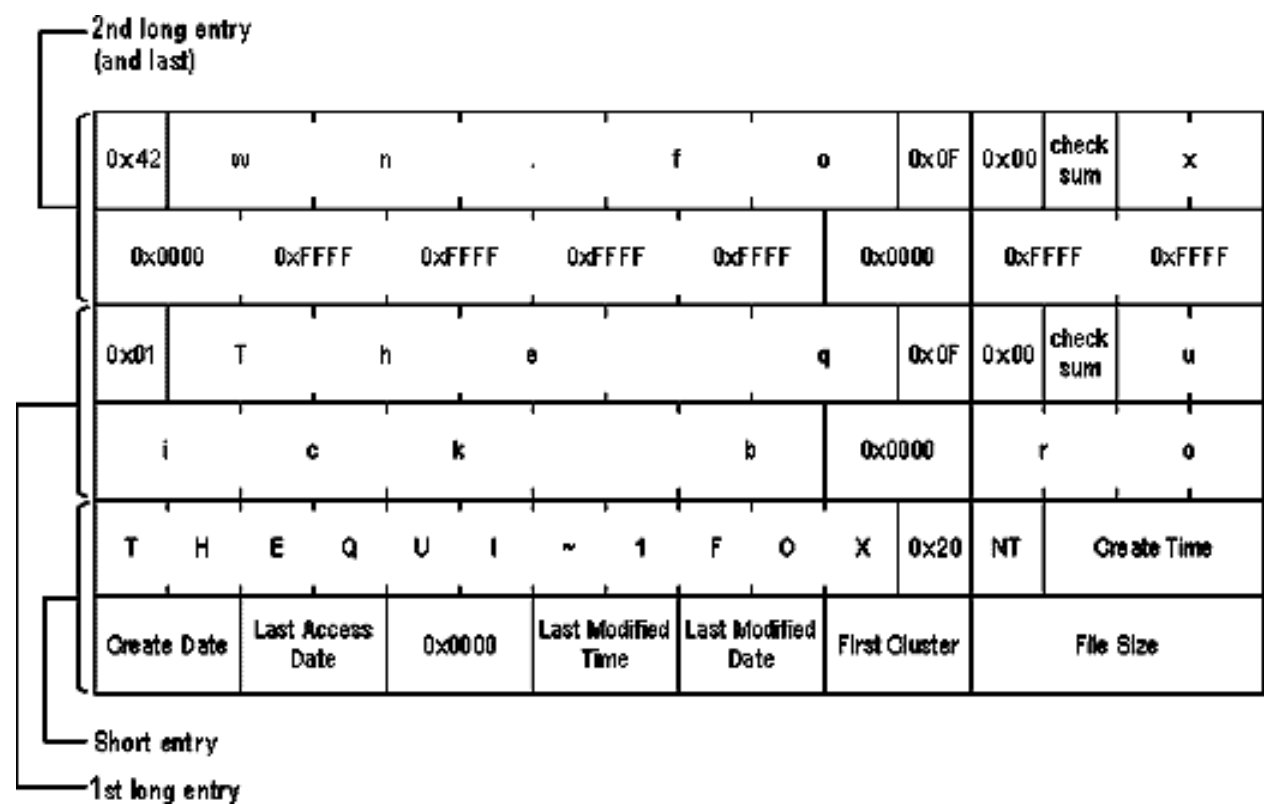
- 目录是目录项的顺序文件(即大小相同的记录序列), 不对目录项排序
 - 若目录中包含的文件数目较多, 则搜索效率低。
 - 每个目录项大小为32字节, 其内容包括: 文件名(8+3个字符), 属性(包括文件、子目录和文件卷标识), 最后一次修改时间和日期, 文件长度, 第一个簇的编号
 - 在目录项中, 若第一个字节为E5h, 则表示空目录项; 若为05h, 则表示文件名的第一个字符为E5h
 - 文件名不区分大小写

FAT32文件系统

- FAT32的根目录区(ROOT区)不再是固定区域、固定大小，而是可以看作数据区的一部分，采用与子目录文件相同的管理方式。根目录下的文件数目不再受最多256的限制
- 目录项仍占32字节，可以是文件目录项、子目录项、卷标项(仅根目录有)、已删除目录项、长文件名目录项等。
- 支持长文件名格式
- 支持Unicode
- 无法支持高级容错特性，不具有内部安全特性，因而仍然无法达到高性能文件系统的要求

FAT32文件系统

■长文件名目录项：例如，文件名为The quick brown.fox，采用Unicode编码，则FAT32的目录项为



NTFS文件系统

- NTFS为改进的多级目录结构，支持文件别名
- NTFS文件由多个文件属性构成，每个属性由属性名和属性流(stream, 简单字节队列)组成；用户可自定义属性
- NTFS支持用户权限管理：
 - 有5种权限划分：读、写、运行、删除和修改权限
 - 支持按用户、用户组分配权限
- NTFS文件支持数据压缩功能
- NTFS卷结构支持容错功能

NTFS文件系统

- 卷建立在磁盘分区上
- 一个磁盘可以有多个卷，一个卷也可以由多个磁盘组成
- 卷上的数据可分为
 - 元数据
 - 用户数据

NTFS文件系统

- 簇作为磁盘空间分配和回收的基本单位
- 簇的大小是用户在使用Format命令或其他格式化程序格式化卷时确定的。簇因子随卷的大小而不同，但都是物理扇区的整数倍，通常是2的幂

卷大小	每簇的扇区	缺省的簇大小
小于等于512MB	1	512字节
513MB~1024MB(1GB)	2	1024字节(1KB)
1025MB~2048MB(2GB)	4	2048字节(2KB)
大于等于2049MB	8	4KB

NTFS文件系统

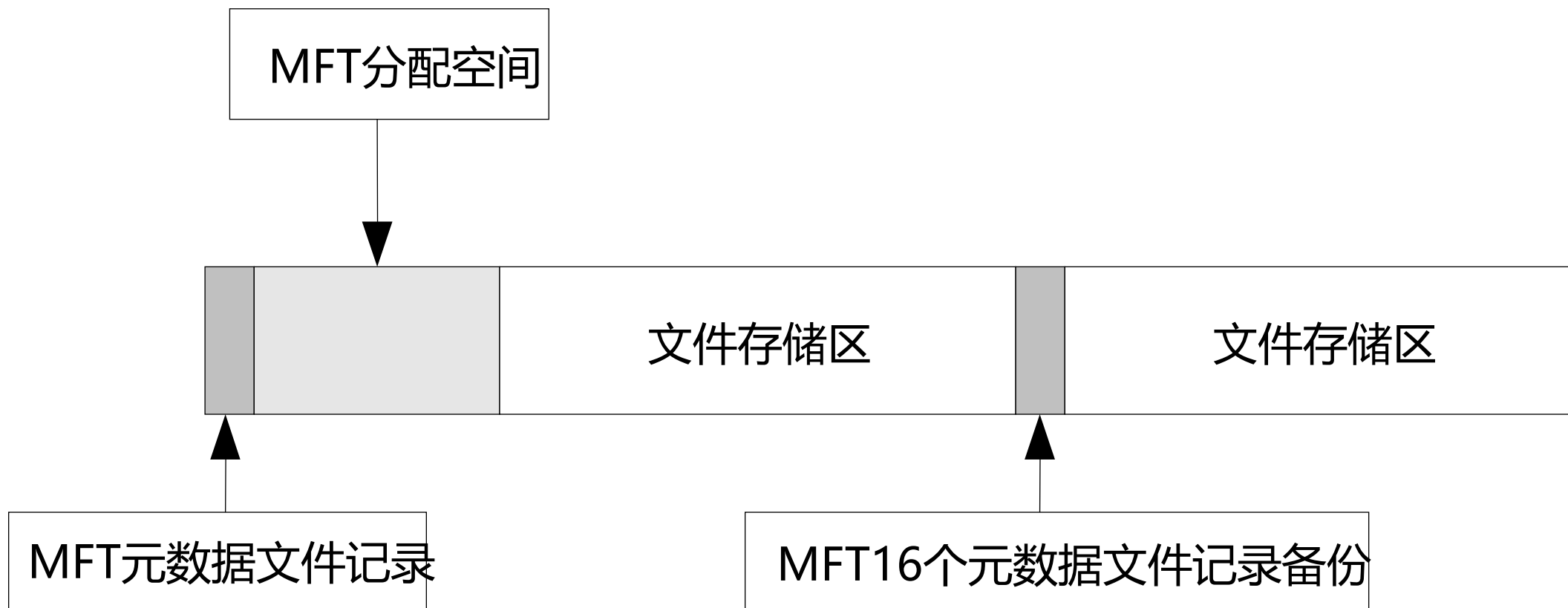
- NTFS使用LCN(Logical Cluster Number, 逻辑簇号)和VCN(Virtual Cluster Number, 虚拟簇号)来进行簇的定位
- LCN是对整个卷中所有的簇从头到尾所进行的简单编号
- VCN是对属于特定文件的簇从头到尾进行编号, 以便于引用文件中的数据

NTFS文件系统

- MFT(Master File Table, 主控文件表)是NTFS卷结构的核心, 是NTFS中最重要的系统文件, 包含了卷中所有文件的信息
- MFT是以文件记录数组来实现的, 每个文件记录的大小都固定为1KB。卷上的每个文件(包括MFT本身)都有一个MFT记录
- MFT中开始的16个记录是保留的, 留给系统的16个元数据文件使用。在MTF中只有这16个元数据文件的记录占有固定的位置。每个这样的元数据文件都有一个以"\$"开头的文件名称, 不过该符号是隐藏的。16个元数据文件的记录之后则是普通的用户文件和目录

NTFS文件系统

■ 主控文件表



NTFS文件系统

- 每个文件都有一个64位的、称为文件引用号(File Reference Number)的唯一标识
- 文件引用号由两部分组成：
 - **文件号**——文件号为48位，对应于该文件在MFT中的位置
 - **文件顺序号**——文件顺序号随着每次文件记录的重用而增加，这是为了让NTFS进行内部一致性检查

NTFS文件系统

- NTFS文件是属性/属性值的集合。文件数据就是未命名属性的值，其他文件属性包括文件名、文件拥有者、文件时间标记等
- 每个属性由单个的流(stream)组成，即简单的字符序列
- NTFS提供对属性流的各种操作：创建、删除、读取(字节范围)以及写入(字节范围)
- 读写操作一般是针对文件的未命名属性的，对于已命名的属性则可以通过已命名的数据流句法来进行操作

NTFS文件系统

- NTFS文件是属性的集合，通常所说的文件内容是指未命名数据属性流
- 例：定义两个数据属性：
ntfile(数据)和ntfile:data(自定义数据)。在向非NTFS文件系统复制时，自定义属性会丢失

```
echo test....data >ntfile
echo test....user defined data >ntfile:data
more <ntfile
more <ntfile:data
copy ntfile e:ntfile (E:为NTFS文件系统)
copy ntfile g:ntfile (G:为FAT文件系统)
more <e:ntfile
more <e:ntfile:data
more <g:ntfile
more <g:ntfile:data
```

NTFS文件系统

- 当属性值能直接存放在MFT中时，该属性就称为常驻属性(resident attribute)。有些属性总是常驻的，这样NTFS才可以确定其他非常驻属性。例如，标准信息属性和索引根就总是常驻属性
- 如果属性值能直接存放在MFT中，那么NTFS对它的访问时间就将大大缩短
 - 小文件或小目录的所有属性，均可以在MFT中常驻。小文件的未命名属性可以包括所有文件数据。小目录的索引根属性可以包括其中所有文件和子目录的索引
 - 大文件或大目录的所有属性，就不可能都常驻在MFT中

NTFS文件系统

- 如果一个属性(如文件数据属性)太大而不能存放在只有1KB的MFT文件记录中，那么NTFS将从MFT之外分配区域。这些区域通常称为一个延展(run)或一个延伸(Extent)，它们可用来存储属性值，如文件数据。如果以后属性值又增加，那么NTFS将会再分配一个延展，以便用来存储额外的数据。值存储在延展中而不是在MFT文件记录中的属性称为非常驻属性(nonresident attributes)
- 在标准属性中，只有可以增长的属性才是非常驻的。对文件来说，可增长的属性有数据、属性列表等。标准信息 and 文件名属性总是常驻的

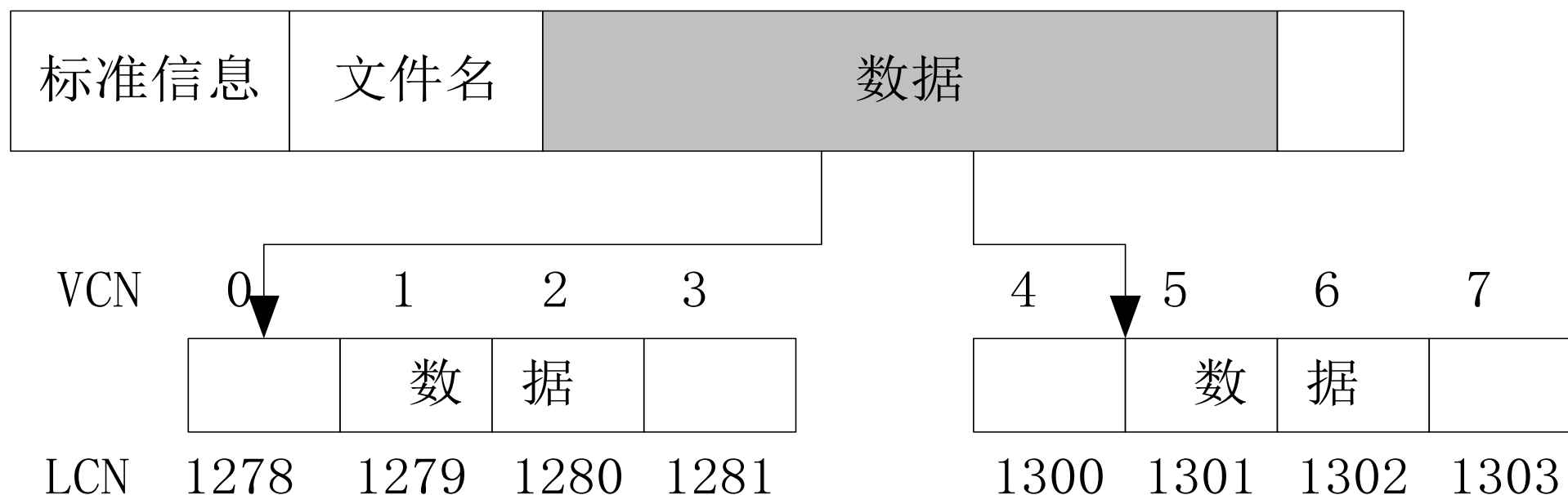
NTFS文件系统

■常驻属性与非常驻属性

标准信息	文件名	文件索引			空
		文件1	文件2	文件3	

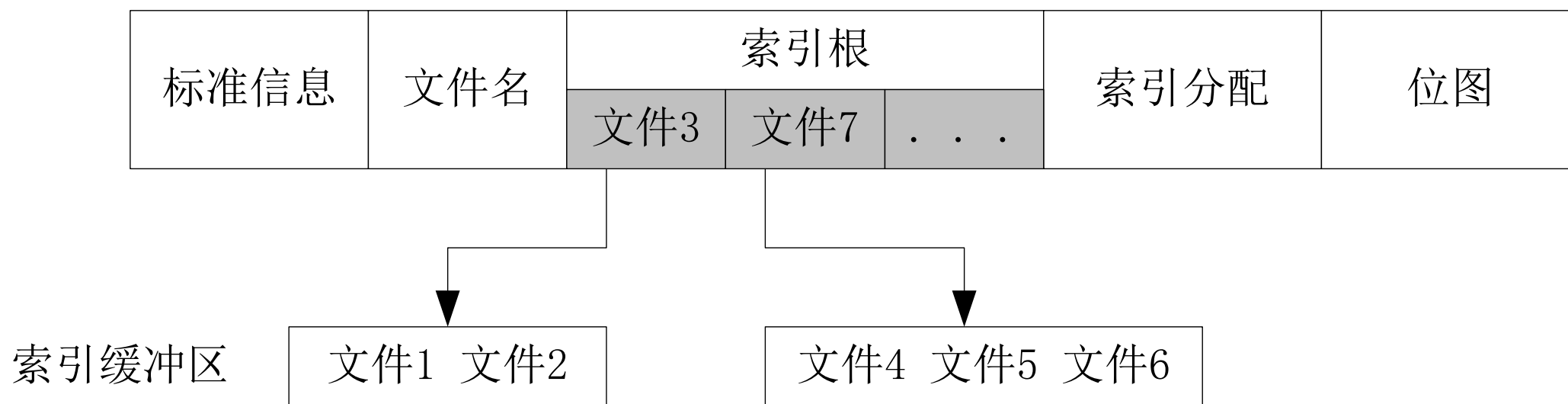
NTFS文件系统

■常驻属性与非常驻属性



NTFS文件系统

■常驻属性与非常驻属性

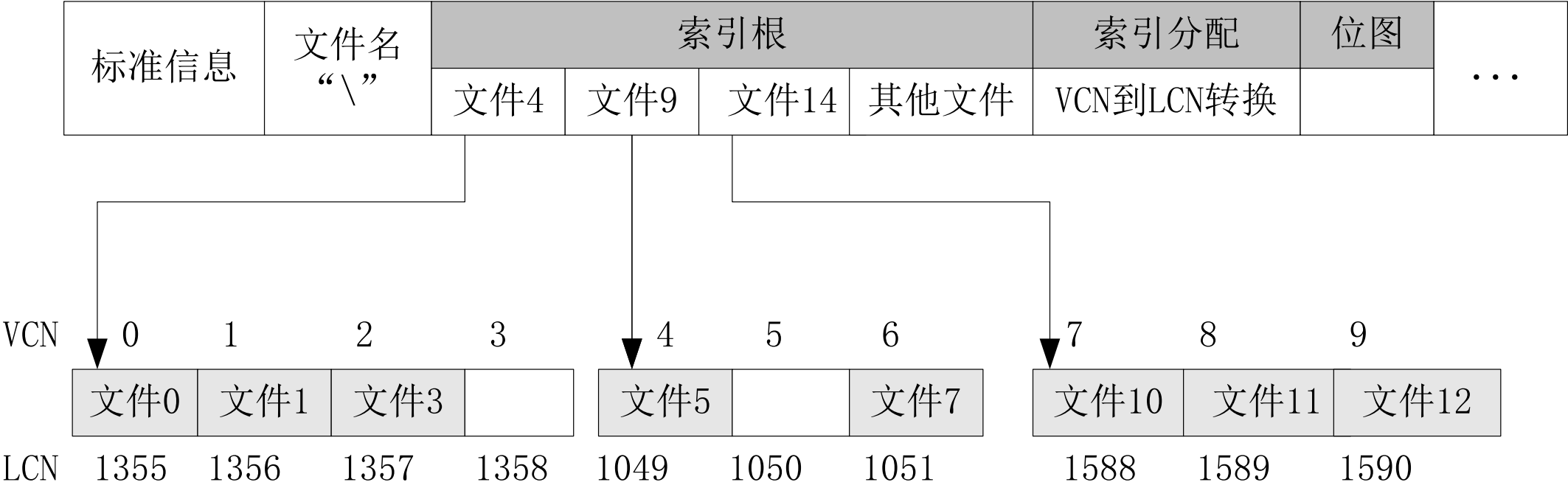


NTFS文件系统

- 索引根属性将相应目录中的文件名和子目录名进行排序
- 对于一个大目录，文件名实际存储在组织文件名的固定4KB大小的索引缓冲区中
- 索引缓冲区是通过B+树数据结构实现的
- 索引分配属性包含了索引缓冲区的VCN到LCN映射
- 位图属性跟踪在索引缓冲区中哪些VCN是在使用而哪些是空闲的

NTFS文件系统

索引根



数据压缩

标准信息	文件名	开始的VCN	开始的LCN	簇数
		0	1200	16
		32	1280	16
		64	1356	16
		80	967	16

■压缩稀疏文件

- 稀疏文件是指相对于文件大小而言只有少量非零数据的文件
- NTFS只给那些包含非零数据的延展分配磁盘空间
- 当程序从压缩文件中读取数据时，NTFS通过检测该位置是否有VCN到LCN的映射来决定该数据是不是零数据
 - 若有映射，则为非零数据，需要从磁盘上读取
 - 若没有映射即存在尚未分配的“空洞”(unallocated hole)，则为零数据，就直接返回零数据

数据压缩

- 压缩非稀疏文件
- NTFS是以16个簇为压缩单元来进行一般文件的压缩的
- 当NTFS向压缩文件写数据时，它确保每个延展都以一个虚拟16簇边界开始。因此每个延展中VCN都是以16的倍数开始的，并且延展的长度不大于16

标准信息	文件名	开始的VCN	开始的LCN	簇数
		0	1200	4
		16	1280	8
		32	1356	9
		48	967	16

EXT文件系统

- Linux最初使用MINIX文件系统，只能支持最高64MB硬盘和14字符文件名
- EXT:
 - EXT(EXTended File System, 扩展文件系统)
 - 1992年4月推出，为专为Linux的设计的文件系统
 - 文件容量上限为2GB，允许最长255 字符的文件名
 - 采用inode记录文件的权限、UID、GID以及指向数据块的指针
 - 采用链表追踪空闲空间，容易导致碎片，影响性能

EXT文件系统

■ EXT2:

- 1993年推出
- 作为Linux可扩展的、功能强大的文件系统而设计，具备了高级文件系统的特点
- 引入了块组的概念，将数据和inode尽可能聚集存放，减少了磁头寻道时间
- 抛弃了链表，改用inode位图和块位图跟踪文件和空闲空间，缓解了碎片化问题
- 支持最大4TB 的文件系统，单个文件最大可达2GB

EXT文件系统

■ EXT3:

- 2001年从2.4.15版开始引入到Linux内核开发主线中
- EXT3和EXT2文件系统在硬盘布局上相同，其差别是EXT3文件系统在硬盘上多出了一个特殊的inode(一个特殊文件)，用来记录文件系统的日志
- 支持最大16TB的文件系统，单个文件最大支持2TB

■ EXT4:

- 2008年自 2.6.28 开始正式支持新的EXT4文件系统
- EXT3的改进版，修改了EXT3中部分重要的数据结构，引入了区段（Extents）概念，可以提供更好的性能
- 支持最大1EB的文件系统，单个文件最大支持16TB

EXT文件系统

- EXT文件系统中的文件保存在数据块中。对同一个EXT文件系统来说，其上所有的数据块大小都是一样的。不同的EXT文件系统的块大小可以不一样，数据块大小实际在建立EXT文件系统时指定，并且作为文件系统的基本参数保存在文件系统中
- 单个文件所占用的空间以数据块为单位分配，因此，如果某个文件的大小为 1025 字节，而数据块的大小为 1024 字节，则该文件将占用两个数据块
- 平均来说，每个文件会浪费半个数据块的空间。为了提高文件系统的访问速度，为了文件系统便于管理，而不得不浪费一些磁盘空间

EXT文件系统

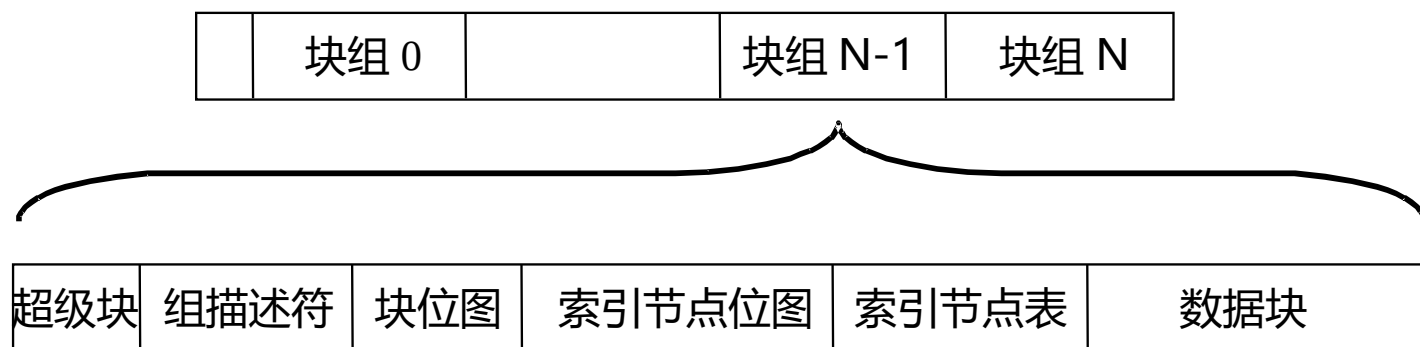
- EXT文件系统利用inode来描述文件系统的拓扑结构。在单个文件系统中，每个文件对应一个inode，而每个inode有一个唯一的整数标识符。文件系统中所有文件的inode保存在inode表中
- EXT文件系统中的目录是一种特殊文件，也有对应的inode，其inode指向的数据块中包含该目录中所有的目录项(文件、目录、符号链接等)，每个目录项对应自己的inode

EXT文件系统的物理布局

- EXT文件系统分布在块结构的设备上
- 文件系统无需了解数据块的物理存储位置，它保存的是逻辑块的编号，由块设备驱动程序完成逻辑块编号到物理存储位置的转换

EXT文件系统的物理布局

- EXT文件系统将逻辑分区划分为块组(Block Group)。每个块组重复保存着对文件系统的完整性非常关键的信息，同时也用来保存实际的文件和目录数据
- 文件系统关键信息的重复存储有助于文件系统在发生故障时还原



EXT文件系统超级块

- 文件系统的超级块(Superblock)包含了文件系统的基本大小和形式。其中包含的数据由文件系统管理程序用来进行文件系统的维护
- 每个块组中包含有相同的超级块信息，但通常只需读取块组0的超级块

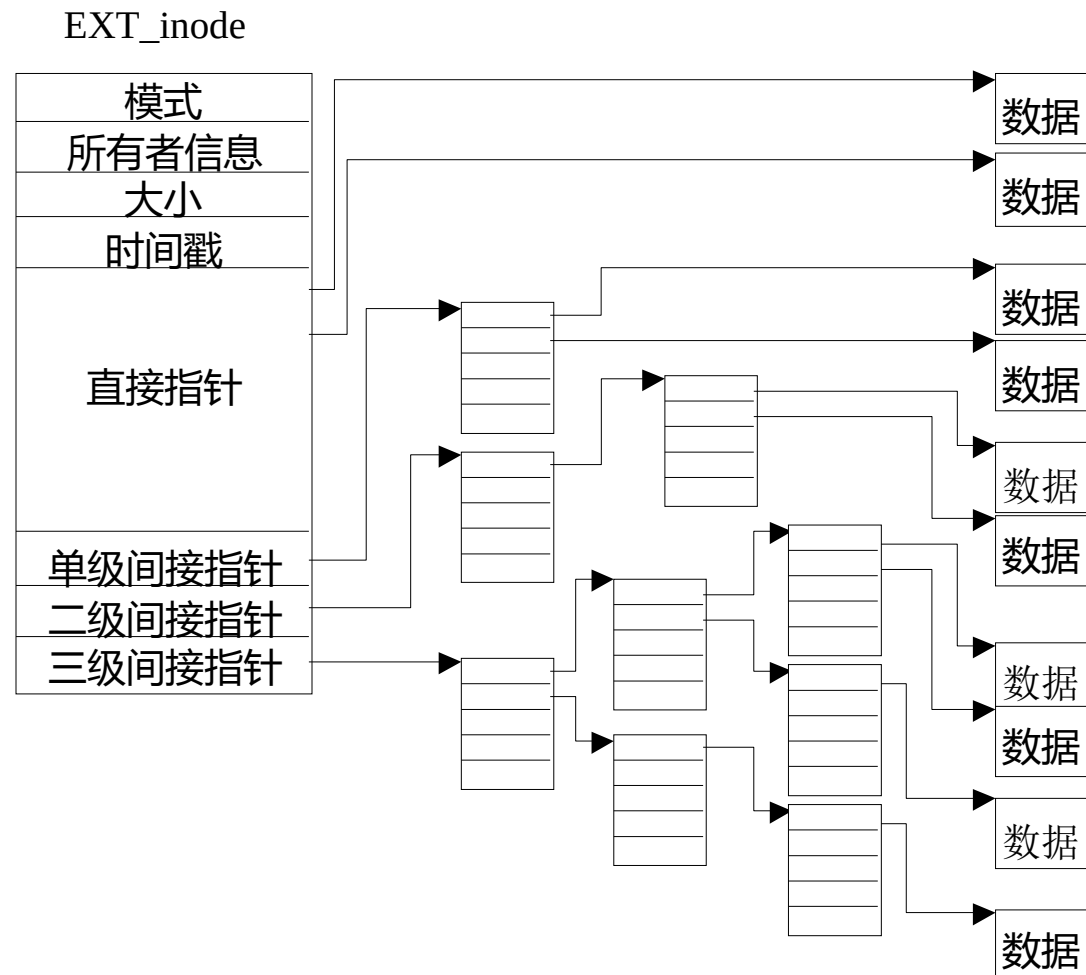
块组描述符

- 每个块组中包含有一个数据结构描述该块组
- 块组描述符保存在每个块组超级块的后面，每个块组中包含所有块组的描述符。块组0中包含的超级块信息和块组描述符是实际使用的数据

数据块位图	包含块组数据块分配位图的数据块编号
索引节点位图	包含块组索引节点分配位图的数据块编号
索引节点表	包含块组索引节点表的起始数据块编号
空闲块计数，空闲索引节点计数，已用目录计数	

节点

- 在EXT文件系统中，索引节点是基本的数据块，每个文件和目录有且只有一个索引节点与之对应
- 每个块组中的索引节点保存在索引节点表中，同时也保存有跟踪节点分配情况的位图
- 前12个数据块指针直接指向包含文件数据的数据块，而其后的三个数据块则是间接指针



EXT目录

- 在EXT文件系统中，目录实际是特殊文件，其中包含了目录项的列表
- 每个目录的前两个目录项始终是标准的"."和"..", 分别代表"本目录"和"父目录"

索引节点	目录项的索引节点
名称长度	目录项名称的长度，以字节为单位
名称	目录项名称

EXT目录

- 当用户需要打开某个文件时，首先要指定文件的路径和名称，根据路径和名称，文件系统可搜索到文件的对应索引节点，从而可读取文件中的数据
- 假定要搜索文件 /usr/hbma/mbox，在 EXT中，该文件的定位过程

6 号索引节点 /usr 的 索引节点内容	132 号数据块 /usr 的 目录项列表	19 号索引节点 /usr/ hbma 的 索引节点内容	406 号数据块 /usr/ hbma 的 目录项列表																																																							
root 目录																																																										
<table><tr><td>1</td><td>.</td></tr><tr><td>1</td><td>..</td></tr><tr><td>4</td><td>bin</td></tr><tr><td>7</td><td>dev</td></tr><tr><td>14</td><td>lib</td></tr><tr><td>9</td><td>etc</td></tr><tr><td>6</td><td>usr</td></tr><tr><td>8</td><td>tmp</td></tr></table>	1	.	1	..	4	bin	7	dev	14	lib	9	etc	6	usr	8	tmp	<table><tr><td>模式</td></tr><tr><td>大小</td></tr><tr><td>时间</td></tr><tr><td>132</td></tr><tr><td></td></tr></table>	模式	大小	时间	132		<table><tr><td>6</td><td>.</td></tr><tr><td>1</td><td>..</td></tr><tr><td>19</td><td>hbma</td></tr><tr><td>30</td><td>zhengx</td></tr><tr><td>51</td><td>shenys</td></tr><tr><td>26</td><td>zhaot</td></tr><tr><td>45</td><td>jianghj</td></tr></table>	6	.	1	..	19	hbma	30	zhengx	51	shenys	26	zhaot	45	jianghj	<table><tr><td>模式</td></tr><tr><td>大小</td></tr><tr><td>时间</td></tr><tr><td>406</td></tr><tr><td></td></tr></table>	模式	大小	时间	406		<table><tr><td>26</td><td>.</td></tr><tr><td>6</td><td>..</td></tr><tr><td>64</td><td>project</td></tr><tr><td>92</td><td>private</td></tr><tr><td>60</td><td>document</td></tr><tr><td>81</td><td>photo</td></tr><tr><td>17</td><td>mbox</td></tr></table>	26	.	6	..	64	project	92	private	60	document	81	photo	17	mbox
1	.																																																									
1	..																																																									
4	bin																																																									
7	dev																																																									
14	lib																																																									
9	etc																																																									
6	usr																																																									
8	tmp																																																									
模式																																																										
大小																																																										
时间																																																										
132																																																										
6	.																																																									
1	..																																																									
19	hbma																																																									
30	zhengx																																																									
51	shenys																																																									
26	zhaot																																																									
45	jianghj																																																									
模式																																																										
大小																																																										
时间																																																										
406																																																										
26	.																																																									
6	..																																																									
64	project																																																									
92	private																																																									
60	document																																																									
81	photo																																																									
17	mbox																																																									
/usr 目录的 索引节点号 为 6	132 号数据 块包含/usr 的目录内容	/usr/ hbma 的索引节点号 为 19	406 号数据 块包含 /usr/hbma 的目录内容	/usr/ hbma/mbox 的索引节点号 为 17																																																						

操作系统，2026

谢谢

OPERATING SYSTEM